

Data Structures & Algorithms (Mandatory Hands-on)

Exercise 2: E-commerce Platform Search Function

Big O Notation

Big O notation is a way to measure how efficient an algorithm is. It tells us how the time taken by a program increases when the amount of data becomes larger. It helps programmers choose the best algorithm for solving a problem.

Search Operation Cases

Best Case:

The required product is found immediately. For example, in linear search, if the product is the first element in the array.

Average Case:

The product is found somewhere in the middle of the array.

Worst Case:

The product is the last element in the array or is not present at all.

Java Program

```
EcommerceSearch.java X
package cognizant_DNS;

class Product {
    int productId;
    String productName;
    String category;

    Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }

    @Override
    public String toString() {
        return "ID: " + productId +
            ", Name: " + productName +
            ", Category: " + category;
    }
}

public class EcommerceSearch {

    // Linear Search
    public static Product linearSearch(Product[] products, int id) {
        for (Product p : products) {
            if (p.productId == id) {
                return p;
            }
        }
        return null;
    }
}
```

```
EcommerceSearch.java ×
// Binary Search
public static Product binarySearch(Product[] products, int id) {
    int low = 0;
    int high = products.length - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (products[mid].productId == id) {
            return products[mid];
        } else if (products[mid].productId < id) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return null;
}

public static void main(String[] args) {
    Product[] products = {
        new Product(101, "Laptop", "Electronics"),
        new Product(102, "Mobile", "Electronics"),
        new Product(103, "Shoes", "Fashion"),
        new Product(104, "Watch", "Accessories"),
        new Product(105, "Bag", "Fashion")
    };

    int searchId = 104;

    // Linear Search
    Product result1 = linearSearch(products, searchId);
    if (result1 != null)
        System.out.println("Linear Search: " + result1);
    else
        System.out.println("Product not found.");

    // Binary Search
    Product result2 = binarySearch(products, searchId);
    if (result2 != null)
        System.out.println("Binary Search: " + result2);
    else
        System.out.println("Product not found.");
}
}

Console × IDE 2026-06 Release
<terminated> EcommerceSearch [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe (26-Jun-202
Linear Search: ID: 104, Name: Watch, Category: Accessories
Binary Search: ID: 104, Name: Watch, Category: Accessories
```

Comparison of Linear Search and Binary Search

Linear Search: $O(n)$

- Checks each product one by one.
- Works even if the products are not sorted.
- Takes more time when the number of products is large.

Binary Search : $O(\log n)$

- Divides the search space into two halves repeatedly.
- Requires the products to be sorted.
- Much faster for large amounts of data.

Which Algorithm is Better for an E-commerce Platform?

For an e-commerce platform, Binary Search is more suitable because an online shopping website may contain thousands or even millions of products. Searching every product one by one would take more time. Since Binary Search reduces the search space by half in each step, it provides faster results and improves the user experience. However, the products must be stored in sorted order for Binary Search to work properly.

Exercise 7: Financial Forecasting

What is Recursion?

Recursion is a programming technique in which a method calls itself repeatedly until a stopping condition, called the **base case**, is reached. It helps solve problems by breaking them into smaller and simpler subproblems.

Time Complexity

The recursive method makes one function call for each year.

the time complexity is: $O(n)$

where **n** is the number of years.

The space complexity is also:

$O(n)$

because each recursive call is stored in the call stack.

How to Optimize the Recursive Solution?

For a large number of years, recursion creates many function calls and uses extra memory. To avoid excessive computation, we can:

1. Use an iterative approach (loop) instead of recursion.
2. Use Dynamic Programming (Memoization) to store previously calculated results and avoid recalculating them.

For this problem, an iterative solution is generally more efficient because it uses constant extra space ($O(1)$).

Java Program implementation:



The screenshot displays an IDE with two tabs: 'EcommerceSearch.java' and 'FinancialForecasting.java'. The 'FinancialForecasting.java' tab is active, showing a Java program that implements a recursive method to calculate future value. The code includes a package declaration, a class definition, a recursive method, and a main method with test values. Below the code editor, the console window shows the program's output, which matches the values used in the code.

```
package cognizant_DNS;
public class FinancialForecasting {

    // Recursive method to calculate future value
    public static double predictFutureValue(double currentValue,
                                           double growthRate,
                                           int years) {

        // Base case
        if (years == 0) {
            return currentValue;
        }

        // Recursive call
        return predictFutureValue(
            currentValue * (1 + growthRate),
            growthRate,
            years - 1);
    }

    public static void main(String[] args) {

        double currentValue = 10000; // Initial amount
        double growthRate = 0.10;    // 10% growth rate
        int years = 5;

        double futureValue = predictFutureValue(
            currentValue,
            growthRate,
            years);

        System.out.println("Current Value: " + currentValue);
        System.out.println("Growth Rate: " + (growthRate * 100) + "%");
        System.out.println("Years: " + years);
        System.out.println("Predicted Future Value: " + futureValue);
    }
}
```

Console Output:

```
<terminated> FinancialForecasting [Java Application] C:\Program Files\Java\jdk-25\bin\javaw.exe
Current Value: 10000.0
Growth Rate: 10.0%
Years: 5
Predicted Future Value: 16105.100000000008
```